ChatGPT

Technical Briefing: Web Content Filtering Project

by Dinis Cruz and ChatGPT Deep Research – 2025

Introduction and Project Overview

This technical briefing outlines the architecture and design of the **Web Content Filtering Project**, which aims to give users fine-grained control over the content they see as they browse the web. The project's core idea is to intercept and dynamically modify web pages in real-time, allowing unwanted content to be filtered out and relevant content to be highlighted. By doing so, users can personalize their web experience – for example, hiding negative news or showing only articles about their favorite sports teams – all **without requiring changes from the websites themselves**.

This capability is aligned with emerging trends in generative AI and semantic web technology. It leverages recent advances in **Large Language Models (LLMs)** and **knowledge graphs** to achieve functionality that would previously have required enormous engineering effort. In essence, we use LLMs to interpret and label web content, then represent both the content and the user's interests as semantic graphs [1]. By bridging these graphs with deterministic algorithms, the system can filter or transform content on the fly in a transparent and explainable way. Crucially, once the initial AI-driven analysis is done, the browsing experience involves **no inline LLM calls** – meaning after the first run, pages load quickly using cached decisions and do not depend on expensive AI processing for each view.

We will implement several Minimum Viable Products (MVPs) to validate this technology. One MVP will focus on **sentiment-based filtering** on a news site (e.g. the BBC News homepage): certain headlines or snippets deemed "negative" in tone can be blacked out or removed, giving the user a positivity-filtered news feed. Another MVP will provide **topic-based customization** on a sports news page: for example, a user interested only in football (soccer) and basketball – and specifically following teams like Wrexham and Benfica – will see those stories normally, while all other sports news is hidden or muted. These scenarios are both **practical and personally relevant**, demonstrating everyday use-cases for the content filtering technology.

In the following sections, we detail the system's design principles, the end-to-end processing pipeline, the technical components (including open-source tools and past research contributions being leveraged), and how we ensure performance, determinism, and explainability. Throughout, we will reference prior work by Dinis Cruz on GenAI pipelines, semantic graphs, and content provenance, as this project builds directly on those foundations.

Key Design Principles

Before diving into the implementation, it's important to understand the guiding principles of our approach:

- **Personalization via Semantic Graphs** – At the heart of the system is the idea of representing both **content** and **user preferences** as structured knowledge graphs. Instead of treating a web page as just text, we interpret it as a collection of facts or attributes (for example, an article might have topics like "Premier League", "Liverpool FC", and sentiment "negative"). Similarly, a

user's interests (or a filtering profile) are represented in a graph structure (e.g. nodes for "Football", "Basketball", "Benfica", etc.). Using graphs allows us to match content to user profiles by finding shared nodes or relationships, rather than brittle keyword matching. As noted in related research, **modeling content and user profiles as semantic data graphs and then using LLMs to bridge and enrich these graphs** is a powerful approach for personalization [1].

- **Minimal In-Line LLM Usage** – One of our core goals is to make the *browsing experience* fast and reliable. Large Language Models are used in our pipeline to analyze and annotate content, but **they are not used in real-time as the user loads each page**. All heavy LLM processing is done either ahead-of-time or on the first encounter with new content. The results of those LLM calls (which might take a few seconds) are cached. Thereafter, showing a filtered page involves only lightweight operations (graph lookups and HTML manipulation) that are extremely fast. This design ensures we don't introduce long delays or unpredictability when serving pages to the user. In practical terms, our target is that **a never-seen page might take up to ~10 seconds** to process (mostly due to one-time LLM calls), but **subsequent loads of the same page (or portions of it) will incur perhaps a few hundred milliseconds** of overhead at most. Essentially, we start with AI in the loop to classify content, but we end with straightforward rules – *"No LLM in-line."*

- **Deterministic and Reproducible Results** – Given the same input (a web page) and the same user profile, the system should always produce the same filtered output. This determinism is important for trust and debugging. Unlike a pure end-to-end AI system that might give slightly different answers each time, our multi-step pipeline is designed so that once content is classified and stored, the filtering decision is a simple deterministic graph query or rule evaluation. Part of achieving this determinism is using structured outputs from LLMs and rigorous data schemas. We enforce type-safe schemas for LLM outputs using techniques from the OSBot TypeSafe framework, which defines Python classes to mirror the expected JSON structure. The LLM's response is parsed into these classes, ensuring the data conforms to the expected format [2]. This is analogous to how one might use Protocol Buffers or JSON schemas in a traditional pipeline, and it prevents the "black box" effect – every piece of data in the pipeline is structured, validated, and traceable.

- **Provenance and Explainability** – A major advantage of our approach is that **every decision made by the filter can be explained and audited**. Because we break down the content filtering into discrete steps and because we store the outputs of each step, we can always dig in and see why something was filtered out or left in. The system maintains a provenance trail linking content to the reasons for its inclusion/exclusion. For example, if a certain news article is removed for being "off-topic", we will have data showing that the article's topics (say, "Celebrity Gossip") did not match any of the user's interests, or if a paragraph is hidden for negativity, we'll have a sentiment score or label attached to that paragraph. This level of transparency is rarely possible in monolithic AI filtering solutions. In fact, Dinis Cruz's work on deterministic GenAI pipelines (such as the MyFeeds.ai project) demonstrated that by capturing each transformation's output, one can answer user questions like *"Why am I (not) seeing this?"* with concrete evidence [3]. For instance, MyFeeds could explain a recommendation by saying *"This article is shown because it mentions GraphQL and your profile lists GraphQL as an interest"* [3]. Our content filter will provide similar explanations for removed content (e.g., "hidden because it's not about your selected topics" or "hidden due to negative sentiment"), backed by data.

- **Performance via Caching and Incremental Updates** – To reiterate, performance is key for a good user experience. We treat our storage (an S3 bucket or local file system) as a database of knowledge about pages we've seen. Everything from raw HTML to extracted text to semantic

classifications is saved, so we **never process the exact same content twice**. We use content hashes to identify when two pages or sections are identical. If a user visits the BBC homepage now and again in an hour, and only 10% of the content changed, we will only run AI analysis on that 10%. All previously seen parts are retrieved from cache. This approach – treating the file system or object store as a first-class database – follows a **LETS philosophy (Load, Extract, Transform, Save)** that Dinis has championed [4]. It leverages the fact that cloud storage like S3 is cheap, versionable, and easily queried, thus serving as a convenient persistent memory of our pipeline's state [4]. We will save not just final results, but intermediate states as well, enabling reuse and "time travel" debugging (we can reconstruct how a page was processed at a given time by loading the saved intermediate files).

- **Seamless Integration (Man-in-the-Middle Proxy)** – From a deployment standpoint, we want this system to work without requiring any modifications to user behavior beyond a one-time setup. The user will either install a local application or configure their browser to use our **proxy**. This proxy acts as an HTTPS man-in-the-middle (with the user's permission, essentially like a benevolent content filter). It intercepts web page requests and responses. When the user requests a page, the proxy fetches the page from the origin server, then performs the filtering transformations before the content reaches the user's browser. The origin web server is none the wiser – it sees a normal request and sends a normal response. All the magic happens in our proxy. This is important for broad compatibility: any website can be filtered without cooperation, and users can keep their preferred browser. (In later phases, we might also offer a browser extension or other integration, but the proxy approach is our starting point for flexibility.)

- **User Awareness and Control** – Especially in early demos and testing, we plan to make the filtering overt. The proxy will inject a small banner or indicator at the top of pages it filters, informing the user that content has been modified. This serves multiple purposes: it's a visual confirmation that the filter is active (useful for debugging when toggling the system on/off), it's a reminder about privacy (the user knows the page content was intercepted and processed), and it provides a UI hook for interactive features (for example, clicking the banner could reveal which content was removed and allow the user to toggle filters or view the provenance explanation). Eventually, in a production setting, such a banner could be optional or more subtle, but during development it will be invaluable for transparency. It also underscores a philosophy of **empowerment, not deception** – the user should always know when AI has altered their content, and they should be able to find out why and reverse it if desired.

With these principles in mind, let's walk through the actual workflow of how a web page goes from the origin server to a filtered result in the user's browser, detailing each technical step in the pipeline.

End-to-End Pipeline Workflow

The content filtering pipeline can be thought of as a series of transformations on the data (similar to an ETL pipeline, but for web content). We can label the stages as **Load**, **Extract**, **Transform**, and **Save** (mirroring the LETS methodology). Here's the step-by-step breakdown:

1. Page Load via Proxy (Intercept and Save Raw Content): When the user navigates to a webpage (say `https://www.bbc.com/` for the BBC News homepage), the request is routed through our proxy. The proxy (which runs either locally on the user's machine or on a server as a cloud service) forwards the request to the real website and fetches the page. When the response (HTML content) comes back, the proxy first **saves a copy of the raw HTML** to persistent storage (for example, an AWS S3 bucket or a local directory). This stored copy is indexed by URL and timestamp. We effectively create a **database of**

web pages we've seen, where each page's content is stored in a timestamped folder (this allows versioning – we can keep snapshots of how a page changes over time). The idea of storing raw content immediately is borrowed from our content capture architecture, where separating the capture from processing is valuable. As described in a related project document, *"the captured content is then sent to a stateless backend and stored in AWS S3… preserving raw web content with minimal server-side processing, enabling future analysis such as semantic knowledge graphs or provenance checks"* [5]. By hashing the content and using it as an identifier, we also avoid storing duplicates – if the same content was saved previously, we can just reference the existing record [6]. At this point, we return the **HTML content** (unmodified) to the pipeline for further processing, but note that we haven't sent anything to the user's browser yet – the response is being held and transformed within our system.

2. Parsing HTML to a Typed Structure: Next, the raw HTML is parsed into a structured form that our code can easily manipulate. We utilize the **OSBot TypeSafe** framework for this. OSBot (an open-source toolkit developed by Dinis Cruz) has utilities to work with web content and define type-safe models. We create a representation of the page's Document Object Model (DOM) as Python objects – essentially mirroring the HTML hierarchy (elements, attributes, text) in a Python class structure. Each HTML element becomes a node/object with properties (tag name, attributes, parent/children relationships). Text within elements is captured as separate text nodes. By using **TypeSafe classes**, we ensure that all elements and their relationships are strongly typed and validated, which prevents errors down the line. This step is crucial because it moves us from dealing with raw text (fragile string parsing) to dealing with **in-memory objects** that can be traversed and analyzed logically. It's similar to how a browser itself parses HTML, but here we have our own representation we can work with in Python. According to the LETS pipeline principles, we then **save this parsed DOM structure** (for example as a JSON file or a serialized Python pickle) to our storage. This might be stored as something like `page_dom.json`. By persisting it, we can skip this parsing step on future runs if the same raw HTML is encountered. It also gives us an artifact to inspect for debugging.

Technical note: OSBot's TypeSafe system was designed to allow easy conversion between JSON and Python objects, and it leverages Pydantic or similar libraries under the hood for (de)serialization [7]. This means after parsing, we could dump the Python DOM object to a JSON file, and later reload that JSON into the same Python object model, guaranteeing consistency. Using a type-safe model for the DOM also makes it easier to traverse and query (we can search for elements of a certain type, or find text in certain sections, with proper tree relationships).

3. Converting DOM to Graph Representation: With a typed DOM in memory, the next step is to convert this into a graph data structure. We use **MGraph-DB (MGraph-AI)**, an open-source memory-first graph database library, to construct a graph where each node corresponds to a piece of content (e.g., an element or text segment) and edges represent relationships (e.g., "X is child of Y" in the DOM tree). At first, this graph is basically a representation of the HTML structure itself – think of it as the DOM tree turned into a graph of nodes (div, span, p, img, etc.) with parent-child edges. We pay special attention to text nodes: when parsing, we introduced a slight augmentation to the DOM model by creating explicit **Text nodes** for stretches of text, rather than leaving them as just string values. Each Text node is linked to its container element. This way, every piece of textual content in the page is an addressable node in our graph.

Once the graph is built, we **persist it to storage** as well (e.g., `page_graph.json`). MGraph-DB allows us to serialize the graph to JSON easily, since it was designed to treat JSON as the storage format for graphs. In fact, MGraph's design decision was to keep graphs in memory during computation for speed, but **persist every update to the file system as JSON, making the file system the source of truth** [8]. This gives us the best of both worlds: the performance of in-memory operations with the reliability and debuggability of persistent storage. Every node and edge can be saved, and we can even diff these

JSON files to see changes over time. At this stage, the graph likely contains a few thousand nodes even for a moderately complex page (every paragraph, link, image, etc., plus each text snippet becomes a node). This is the foundation on which we'll do semantic analysis.

4. Extracting Textual Content: Now that we have the full page graph, we extract the subset of that graph that is relevant for text analysis. Not all nodes are equal from a content perspective – for filtering decisions we mostly care about textual content (the words on the page). So we perform a **graph query** or traversal to collect all **Text nodes** (nodes representing actual readable text) and, importantly, gather their surrounding context (such as their parent element or section). The result of this is essentially a list (or subgraph) of textual content pieces. Each item might include, for example, the text string "Manchester United wins FA Cup final" and metadata like: it was inside an `<h2>` headline element within a `<div id="sport-section">`. We then structure this extraction as a new **"Content Items" graph** – where each content item node contains the text and perhaps links (edges) to the section or parent nodes that situate it in the page. We save this content-focused graph (let's call it the **Content Tree** for the page) as another JSON (`page_content_graph.json`). Essentially, we are transforming the raw HTML graph into a distilled form that is easier to feed to an AI and reason about. This transformation is flexible – during development we may tweak what metadata we carry with each text (e.g., we might include the HTML tag hierarchy as part of the content item's properties, or an XPath/CSS selector to locate it on the page when we need to reconstruct). The key is we've isolated all the textual pieces that might be individually classified.

5. LLM Semantic Classification of Content: This is the most **computationally intensive step**, and where the generative AI power comes in. For each textual content node we extracted, we need to classify or annotate it according to the filtering criteria. In our current MVP scope, we have two parallel classification tasks:

- **Sentiment Analysis (Positive/Negative)** – We determine whether the content of the text is positive, negative, or neutral in sentiment. Initially, we might treat this as a binary or ternary classification (positive vs negative, ignoring neutral or handling neutral as "leave it visible"). This could later be extended to a more nuanced scale (e.g., a score from 1 to 5, or a percentage positivity). The purpose here is to allow a user to filter out negative or depressing news, or conversely, filter out overly positive fluff if they so choose. To get this, we send the text to an LLM (like GPT-4 or another model with good zero-shot classification ability) with a prompt asking for the sentiment. Importantly, we use a **structured output prompt** – we ask the LLM to respond in JSON format, indicating the sentiment category (and perhaps a confidence). By using a structured format, we can directly parse the LLM's answer. For example, the LLM might return: `{"sentiment": "negative"}` for a text about an accident, or `{"sentiment": "positive"}` for a text about a victory or good news. If the LLM is verbose or explains its reasoning, our parser will ignore everything except the JSON snippet thanks to how the OSBot TypeSafe schema is set up. Each result is attached to the corresponding content node in the Content graph (we may create an edge or property like "hasSentiment → Negative"). This effectively extends our content graph with a semantic layer (let's call it the **Sentiment Graph**).

- **Topic and Entity Classification (e.g. Sports Category)** – In parallel, we classify content by topic. For the sports use-case, for each text item we want to know: is this sports-related? If so, what sport, and which teams or players are mentioned? We can again leverage the LLM for this by prompting it to extract structured data. For example, we might ask: *"Is this text sports-related? If yes, identify the sport and any team names or athlete names mentioned. If not, say it is not sports-related."* The LLM could output: `{"sport": "football", "teams": ["Manchester United", "Manchester City"]}` for a headline about a football match, or `{"sport":`

`null}` for an article unrelated to sports. We would design a schema for this output (maybe a class `SportsClassification` with fields for `sport` and `teams[]`, where `sport` could be an enumerated type or a string). Again, by using a structured approach, we get a reliable JSON that we parse into our graph. Each content node then gets linked into a **Topic Graph** – e.g., the node for "MU wins the final" would have an edge connecting it to a node representing "Football" (because that's the sport category identified), and perhaps edges to team nodes "Manchester United" and "Manchester City" in an ontology of sports entities. We may maintain a taxonomy/ontology graph of sports terms (so that "football" is understood as a type of "Sports" content, and "Manchester United" might be a child node under "Football > Teams > Premier League"). The initial taxonomy might be simple and hard-coded, but as this project grows we can refine it. (This touches on the idea of evolving ontologies – initially we might let the LLM output whatever category words it wants, then later normalize them or constrain them to a controlled vocabulary. Research suggests that starting with free-form extraction and then iteratively enforcing a cleaner ontology is a practical approach [9], which mirrors how MyFeeds moved from raw LLM outputs to a more explicit set of entity types with human feedback.)

It's worth noting that this architecture easily generalizes to other classification dimensions. For example, we could also classify each content piece by **topic domain** (politics, entertainment, technology, etc.), by **geographic relevance** (mentions of countries or cities), by **factuality or source credibility**, and so on – depending on future filtering needs. Each would be another property or subgraph attached to the content nodes, derived from LLM analysis. The modularity of having multiple small LLM calls (one per content piece, possibly batched, and specialized per property) follows the principle of using the right tool for each task, rather than one giant prompt that tries to do everything.

All results from this stage are **saved**. For instance, we'll have files like `page_content_sentiment.json` and `page_content_topics.json` or a combined `page_semantic_graph.json` that stores the classified graph. Each piece of text content now has machine-understandable annotations: sentiment label, topic category, and associated entities (sports/team names in our use case). This completes the **Extract** phase (we've extracted knowledge from raw text) and sets the stage for the **Transform** phase, where we actually decide what to show or hide.

6. Building the User's Persona/Preference Graph: Equally important to analyzing the content is understanding the user's preferences – i.e. the filter criteria. We capture this in a **Persona Graph** for the user (or for a given filtering mode). A persona graph is essentially the mirror of the content semantic graph, but for the user's interests. For example, if the user only cares about **Football and Basketball** (and within football, specifically the **Wrexham** and **Benfica** teams), we will create a graph that has nodes for "Football" and "Basketball" (maybe as sub-nodes of a generic "Sports" interest), and child nodes for "Wrexham" and "Benfica" under football > teams (and perhaps any other specific teams/players the user follows). If the user also indicated they want **only positive news**, that preference might be encoded as a node or property in their persona graph (e.g., a node labeled "Prefers Positive Content" or simply a rule that negative sentiment = not relevant to this persona). The persona graph can be constructed manually from user input (say, a settings UI where they check boxes of interests) or it can be semi-automated. In future iterations, we could use an LLM to help expand a user's description into a richer graph – for instance, if a user says *"I'm interested in Portuguese football"*, an LLM could infer that likely teams of interest might include Benfica, Porto, Sporting, etc., and add those. This is similar to how persona profiles were created in the InsightFlow project: *"take a list of interest keywords and use an LLM to expand them into a richer graph of related concepts"* [10] [11]. In our first version, we will keep it simpler and directly represent what the user specifies, but we keep the door open for LLM-assisted persona building, which can surface non-obvious interests (e.g., linking "fintech" to related concepts like "blockchain" or, in sports, linking "La Liga fan" to specific clubs in that league the user didn't explicitly list) [11] [12].

The persona graph is stored similarly in JSON (e.g., `user_profile_graph.json`). It might have a structure parallel to the content ontology. For instance, it could mirror the sports taxonomy so that matching can be done via common category names or IDs. If the user is in a certain "mode" (like positivity-filter mode), that could also be represented here as a boolean flag or a mode identifier node.

7. Relevance Mapping – Matching Content to Persona: Now comes the decisive step – determining which pieces of the page content are relevant to the user (and thus should be shown) and which are not (to be filtered out). We have all the ingredients: a semantic graph of the page's content (with labels like sentiment and topics on each text node) and the user's persona graph (with labels of what they care about). **The simplest form of matching is to check for overlaps between these graphs.** In practice, this means for each content item, we ask questions like:

- Does this content item's topic tag (or entity) appear in the user's interest graph? For example, if a headline is tagged with `sport: football` and `team: Benfica`, and the user's interests include Benfica, that's a direct overlap – this item matches the persona.
- Conversely, if an item is tagged as football but the user only cares about basketball, that item is not relevant.
- If an item has **negative sentiment** and the user profile says they want to avoid negativity, that item is not relevant regardless of topic.
- If the persona had more complex rules (say the user wants political news *only* if it's positive, otherwise hide), those could be encoded as well (like an interest in "Positive Politics" which would only match items that are political + positive).

We can implement this matching logic in code by traversing the content items and checking their attributes against the persona graph. In many cases, a direct graph algorithm can do this: e.g., **graph intersection** (find nodes in content graph that have a relationship to any node in persona graph). Since we gave content nodes explicit links to topic entities (like an edge from a content node to the "Football" node in a taxonomy), and the persona graph likely has a "Football" node if the user cares about it, finding a match can be as easy as seeing if there's a path from the content node into the persona graph. MGraph-DB supports queries and set operations on graphs which we can use to automate this. In some scenarios, we might still use a lightweight LLM prompt to refine the matching – for instance, to evaluate if an article is *strongly* relevant or just tangential. However, the aim is that this step is largely non-ML, purely data-driven. In the MyFeeds project, a similar step was done with an LLM to ensure flexibility with synonyms and context (the LLM could recognize connections that exact graph matching might miss, like mapping "EU regulation on privacy" content to an interest in "GDPR compliance") [13]. For our immediate use cases, synonyms are less of an issue (sports team names are explicit, sentiment is a direct flag), so we can likely do this deterministically.

Regardless of how the matching is implemented, the outcome is a determination for each content piece: *Relevant (keep)* or *Not Relevant (filter out)*. We compile these results into a **mapping result object** – effectively a list of content node IDs that should be shown or hidden. We also record *why* each was classified that way, by linking back to the matching criteria. For example, we might produce a JSON that says:

```
{
  "show": [
    {"node_id": 42, "reason": "Matches interest 'Football' (mentions
Benfica)"},
    {"node_id": 57, "reason": "Positive sentiment content"}
  ],
```

```
  "hide": [
    {"node_id": 13, "reason": "Negative sentiment content"},
    {"node_id": 21, "reason": "Topic not in user's interest (Cricket)"}
  ]
}
```

All this information is stored (e.g., `page_filter_results.json`). Storing the mapping results is important for provenance – it forms the basis of our explanation to the user. In fact, by maintaining the connections between content and persona in a graph structure, we build a **provenance trail** that can justify each inclusion or exclusion [14]. For example, *"Article X is shown because it connects to 3 topics you care about: X, Y, and Z"* (straight from the InsightFlow methodology [15] [14]). In our case, it might be simpler (one topic or one reason), but the concept is the same. This explicit mapping is something we can present to users for transparency.

8. Page Reconstruction (Applying the Filter): Now we have the original page content and a list of what should be visible or hidden. The final step is to **reconstruct the HTML** to reflect the filtering decisions. Since we still have the original DOM/graph in memory (or we can reload it from the saved state), we can go through it and for each content node that was marked "hide", we remove or mask that element. There are a couple of ways to do this. A straightforward approach is: for a text node to hide, we can replace its text with a placeholder (e.g., "[redacted]" or an empty string) in the HTML. Alternatively, we could remove the entire HTML element containing that text (which might be better if we want to collapse space). Initially, we might choose a conservative route like replacing text with a black bar or a note like "[removed]", so the user can see that something was there but is being hidden. For visible items, we may also choose to highlight them (for example, outline the preferred sports news in green). Highlighting is not strictly necessary, but it can be a nice visual confirmation in demo mode that "these are the items of interest". All such modifications are done in the DOM structure and then serialized back to HTML markup.

During this reconstruction, we can also inject the **top banner** we discussed. The banner can be a simple `<div>` at the top of the body that says something like: *"WebContentFilter active: X items removed, Y items highlighted. [View Details]"*. The "View Details" link could point to a local page (perhaps served by the proxy or a static file) that reads the `page_filter_results.json` and displays the provenance information in a friendly way (e.g., a list: *Removed Headline "XYZ" – Reason: Not in Sports interests*). This closes the loop on transparency: the user not only experiences the filtered content but can also inspect exactly what was done.

After injecting the banner and finalizing the HTML, the proxy sends this modified HTML to the user's browser. From the user's perspective, the page loads normally except they notice some content missing and the banner present. The filtering is complete!

To summarize the pipeline in a simple flow: **Browser Request → Proxy fetches page → Save raw HTML → Parse to DOM → DOM to Graph → Extract Text Nodes → LLM classification (sentiment/topics) → Save semantic graph → Load Persona graph → Match graphs for relevant content → Modify HTML (remove/hide content) → Deliver to Browser**. Each of those arrows represents data saved and available for debugging or reuse.

It's worth highlighting how this approach scales and remains maintainable. Each step is separate and outputs files (HTML, JSON graphs, etc.). If something goes wrong in the final output, we can trace back: check the mapping results, check the semantic classifications, check the content extraction, etc. This modular design is inspired by earlier pipeline work. For example, in MyFeeds.ai, the processing was split

into numerous small steps (fetch feeds, extract text, LLM to entities, build graph, compare to profile, etc.), each writing out its result. This made the system **highly debuggable** and resilient: *"if something failed at step 5 for article X, the engineer could retrieve article X's JSON from step 4 and investigate… since each step is idempotent on its input file, the system can retry or resume failed steps without starting over"* [16] [17]. We are employing the same strategy here.

Caching, Performance Optimizations, and Incremental Updates

After the first run of a given page, subsequent visits should be much faster, as the heavy lifting is already done:

- **Content Hashing for Cache Hits:** We identify content pieces by a hash (for example, an SHA-256 of the text content or a combination of text + source URL). These hashes are used as keys in our storage. When processing a page, if a particular text node's hash is recognized (we've seen that exact text before on the same site or another site), we can skip re-sending it to the LLM. Instead, we retrieve the previously stored classification for that text. This is analogous to de-duplicating storage in our content capture project, where each page's content fingerprint is used to avoid storing it twice [6]. Here we avoid *processing* twice. In news sites, the same news blurb might appear on multiple pages (e.g., the homepage and a category page); hashing ensures we classify it once and reuse the result everywhere.

- **"Latest" vs Historical Storage:** We maintain a `latest/` **folder** for each type of artifact (latest version of the BBC homepage content graph, etc.) as well as time-stamped archives (so we can analyze changes over time or roll back if needed) [18] [19]. The system will always refer to the `latest` data for making decisions, but the historical snapshots are invaluable for debugging or explaining time-based differences (e.g., "this story wasn't removed yesterday because it was under a different section which we handled differently").

- **Incremental LLM calls:** If only a portion of a page is new, we only send that portion through step 5 (LLM classification). For example, consider the BBC homepage which updates frequently. At 10:00 AM you load it – we process 100 headlines/snippets. At 10:30 AM, you load again – 10 of those have changed to new stories, 90 are the same. Our system detects the 90 unchanged ones (by hash or by seeing the URL + text of those stories unchanged) and **immediately reuses** the prior classification. It will only call the LLM for the 10 new pieces. This means the 10:30 AM load might finish in, say, 1 second instead of 10, since only a fraction of content needed analysis. In effect, the more you use the system (and especially if many users are using it across overlapping content), the cheaper each new page load becomes. Shared content results in amortized cost.

- **Batching and Parallelism:** On the first load of a large page, we don't actually need to send one content piece at a time to the LLM. We can batch multiple prompts if using an API that allows it, or run multiple LLM requests in parallel (bearing in mind API rate limits and costs). The goal is to utilize the full 10-second budget efficiently. If the LLM can handle, say, 20 classifications in one API call (using a prompt that lists 20 items and asks for JSON outputs for each), we will explore that. We will also parallelize sentiment and topic classification since they are independent – possibly even combine them into one prompt ("for each text, tell me sentiment and sport category/team if any"). However, combining too much in one prompt can complicate parsing and increase error chances, so it's a balance. These optimizations will be tested to meet our performance goals.

- **Client-Side vs Server-Side Considerations:** In early versions, running the proxy and pipeline on the client's machine (with a local LLM or small model) is not feasible for heavy analysis – we rely on cloud-based LLM APIs. Thus, the proxy might be lightweight (just capturing and forwarding data), with the actual processing happening on a backend service or a powerful local daemon. However, as the project evolves, one could imagine moving more logic to the client (for privacy and cost reasons). The architecture is flexible: for instance, the **Pyodide** in-browser approach (running Python in the browser) was used in a related content capture extension to perform hashing and even some testing client-side [20]. For now, our approach is a hybrid: network requests go to our server, which does the heavy computation and returns filtered HTML. We ensure the backend is stateless (all state is in the files/graphs) so it can scale easily (any server handling a request can pull the necessary data from S3). This stateless, serverless-friendly design (where the server is mostly a conduit to storage) is indeed how our content capture backend was structured: *"a stateless FastAPI backend purely as a transient proxy to storage… simply accepts content and writes it to S3… no persistent server database to protect"* [21]. The lack of a traditional database in favor of object storage means we can spin up multiple processing workers in parallel without complex coordination – they all read/write to S3 which serves as the single source of truth.

- **Memory-First Graph Handling:** Thanks to MGraph's memory-file hybrid design, we can keep graphs in memory for fast operations and trust that every change is also on disk for persistence [22]. For example, as we remove nodes or mark them hidden, we can record that in the graph and it's written out. If we had to reload later, we could load the graph of the page and see which nodes were marked hidden last time. This could allow caching of the final **modified HTML** as well. We might even store the fully rendered filtered HTML for a given page-state and persona combo, but since the final render is quick to generate from the graph, it's not strictly necessary.

In summary, the system uses a combination of **caching, content hashing, and splitting work into independent pieces** to ensure that after the initial investment of processing a page, subsequent operations are extremely fast. By treating intermediate results as reusable assets (much like a compiler caches object files), we minimize redundant work. This approach was proven in the MyFeeds.ai pipeline, where each transformation's output was saved and could be reused or inspected, leading to determinism and efficiency [23]. In that project, the result was a personalized content feed that was complex but **behaved deterministically, with every intermediate reasoning step materialized for debugging** [24] – exactly what we strive for here in the context of web browsing.

Technical Components and Tools

This project stands on the shoulders of open-source tools and past research, particularly those developed by Dinis Cruz and collaborators. Here we credit and describe the key components being utilized:

- **OSBot and TypeSafe Schemas:** *OSBot* (OWASP Security Bot) is a framework Dinis has developed over years for automating and integrating security tasks, and it includes a module called **OSBot-Utils TypeSafe** which lets developers define data models in Python that correspond to JSON schemas. We use OSBot's TypeSafe classes to define the structure of our LLM outputs and ensure type consistency [2]. For example, we have a class for the sentiment analysis output and a class for the sports classification output. By registering these with OSBot's system, when the LLM returns JSON, we can load it directly into these classes – if the JSON is missing a field or has a wrong type, we'll catch it immediately. This greatly reduces error-handling code and makes the pipeline robust to LLM quirks. Additionally, OSBot provides wrappers for common web

automation tasks. While our use of it here is mostly for the TypeSafe feature, it's worth noting OSBot can also control headless browsers, interact with pages, etc., which could be useful if we extend the project (for example, to auto-scroll or click to load more content before capturing).

- **MGraph-DB (Memory-First Graph Database):** We rely heavily on *MGraph-DB* (also referred to as MGraph-AI in some write-ups). This is an open-source library (designed by Dinis Cruz) that treats JSON file storage as the backing store for graph data structures [25]. Unlike traditional graph databases which often require running servers or heavy binaries, MGraph is lightweight and can run inside a serverless function or a small script. It keeps the working set in memory but every operation triggers a JSON save, which is perfect for our LETS pipeline approach. MGraph also offers a **type-safe API for graph operations** – meaning nodes and edges can be manipulated via Python classes rather than raw dictionaries, adding to our type safety approach [26] [7]. Some advantages of MGraph for our use case:

- *Ease of merging and diffing:* We can easily merge the content graph and persona graph, or diff them to find overlaps (the overlaps essentially represent relevant content). The JSON outputs can be diffed with standard tools or version control if needed.
- *Visualization:* MGraph can export to Graphviz or other visualization formats [27]. In development, this means we can generate a diagram of a page's content graph or a persona graph, which is immensely helpful for understanding if our extraction logic is working. It also means we could potentially show the user a graph view of how the content matched their interests (though that might be more detail than most would want, it's useful for power users or developers).

- *Memory-footprint and deployment:* Because the entire graph is kept in memory and just saved as JSON, we avoid the need for an external database process. This fits the serverless style where each request can quickly load some JSON, do computations, and save JSON. It's cost-efficient and simplifies deployment (S3 is our database).

- **FastAPI / Backend Infrastructure:** Although not a unique component of the project, it's worth noting we will likely implement the server side using Python and FastAPI (or a similar web framework) to handle requests coming from the proxy. Each request to fetch a page will invoke our pipeline logic. FastAPI makes it easy to integrate with Pydantic (which complements our TypeSafe models) and to structure the asynchronous calls to LLM services. The stateless nature of our approach (with S3 as persistent store) means each request can be handled independently, which is a good match for serverless deployments (we could deploy on AWS Lambda or Azure Functions, for example, scaling out as needed).

- **Generative AI Models:** The choice of LLM is flexible. In development we might use OpenAI's GPT-4 API for its reliable understanding and generation of JSON. We will craft prompts carefully to minimize errors and ensure brevity of responses (since we pay per token and also to reduce parsing complexity). We might also explore open-source LLMs for certain tasks if they can be run efficiently (for example, a local sentiment analysis model or a smaller model fine-tuned on news classification). The system is model-agnostic; as long as we get the JSON in the format we expect, it doesn't matter if it came from GPT-4 or an ensemble of smaller models. We will log which model was used for each classification (that could even be stored in the provenance data for future reference).

- **Related Research and Prior Art:** This project directly builds on ideas from **MyFeeds.ai** (a personalized news feed generator) and **Cyber Boardroom** (an AI-driven advisory system for cybersecurity news) that Dinis has worked on. Those projects validated the approach of multi-phase LLM pipelines with graphs. For instance, the MyFeeds architecture used a series of steps

(often implemented as separate API endpoints) to go from raw RSS feed data to a compiled newsletter, with each step saving its output to S3 [28] [29]. This provided strong provenance and the ability to answer "why is this item recommended?" convincingly [3]. We are essentially repurposing that pipeline concept from push-style feeds to on-demand page filtering. The common thread is the combination of **GenAI + structured data + graphs + provenance**. By citing and crediting these past works, we acknowledge that our filter isn't an isolated invention but the next iteration of a series of ideas. Notably:

- *Deterministic GenAI Outputs with Provenance:* Dinis's talk at OWASP EU and writings [30] emphasize the importance of breaking down AI tasks and capturing each result. Our design follows that to the letter.
- *Graphs of Graphs concept:* In some of Dinis's research, he mentions *"graphs of graphs"* meaning you can have multiple layers of graphs linked together (for example, an ontology graph, a content graph, a user graph) and even evolving ontologies over time [31] [32]. Our filter uses at least two graphs (content vs persona) and effectively links them. While we won't delve into meta-graphs here, the architecture is compatible with very advanced graph-based reasoning if we choose to incorporate it later (for instance, linking out to an external knowledge base graph to get more info on an entity).

In summary, our tech stack is predominantly **Python-based**, with heavy use of **JSON as a lingua franca** between stages, and built on open standards and open-source projects (ensuring we can share parts of this work with the community or integrate improvements from others). By using and crediting these tools (OSBot, MGraph-DB, etc.), we also plan to contribute back by highlighting their capabilities in a new domain (web filtering) and potentially raising issues or extending them as needed for our purposes.

Provenance and Explainability Features

As mentioned earlier, one of the standout features of this project is its ability to explain its own behavior. Here we detail how provenance data is captured and how it might be surfaced to users or developers:

- **Graph-Based Provenance Links:** Every time the system makes a decision to show or hide content, that decision is not just a boolean in isolation – it's derived from a link between a content node and a persona node (or a rule). We explicitly maintain that link. For example, suppose an article headline about "Eagles win championship" is shown because the user is interested in **Basketball**. In the content semantic graph, "Eagles" might be identified as the team name of a basketball team (say the Philadelphia Eagles for argument's sake), and that node is connected to a broader "Basketball" category node. In the persona graph, the user has a "Basketball" node. During the matching step, we find this commonality and mark the item as relevant. We could create a new edge in a **Result Graph** that connects the content item node to the persona interest node "Basketball" and label that edge "matches". If later someone queries *"why was this shown?"*, the system can traverse from the content item to find any outgoing "matches" edges and see what they connect to – in this case it finds "because it matches your interest in Basketball." This is exactly the kind of explanation that builds trust: *"the system can explain: 'Because the article mentions GraphQL and your profile lists GraphQL as an interest'"* in the context of MyFeeds [3]. Replace GraphQL with any topic of interest, and that's our pattern.

- **Deterministic Recalculation:** Because our decisions are stored and based on data, if there's ever a dispute or doubt, we can re-run the logic and get the same answer. If a user says "I think this article should have been included, why wasn't it?", we can take the stored semantic graph of that article and the user's profile and run the matching again. If it still says "not included", we

then inspect: maybe the article was tagged incorrectly (LLM said it's about a sport the user didn't list, but maybe it actually is relevant). If so, we have a few ways to improve: we could adjust the ontology (e.g., alias that sport to something the user likes), or adjust the LLM prompt if it misunderstood something systematically. The provenance data thus not only justifies the present, it guides us to future enhancements.

- **User-Facing Provenance UI:** In the first iteration, the user-facing part might be simple links that open a JSON or a very basic HTML report. But we envision a more user-friendly provenance interface. This could be a pop-up or side panel that, when the user clicks "View Details" on the injected banner, shows a list of content sections that were removed or highlighted, each with a brief reason. For example:

- *Removed: "Climate change causes..." (Negative tone)* – You opted to hide negative sentiment news.

- *Removed: "Cricket: India vs England..."* – You are not following Cricket in your interests.
- *Shown: "Benfica wins 3-0..."* – Matches your interest in Benfica (Football).

The UI could allow the user to toggle a switch next to each reason to say "always/never filter this kind of content". That could provide feedback to the system (for example, if the user toggles "never filter negative news", we'd remove the sentiment rule from their persona).

Another aspect of explainability is to show the **strength of relevance**. In some systems like InsightFlow, they computed a relevance score (e.g., 8.5/10) and could cite multiple matches (article X is 8.5 relevant because it hits 2 big topics you care about) [15] [33]. We can incorporate a simple scoring (e.g., +1 for each interest matched, -1 for each filter criterion failed) to give a sense of confidence. But for transparency, listing matches is probably sufficient. The graphs we store essentially contain all the raw info needed to compute such a score or explanation.

- **Auditing and Logging:** On the backend, every time the filter runs, we will log the steps and results. This log (which can be just console logs or a structured log file) is useful for development debugging. But it also serves as an audit trail: we can later analyze how often the LLM classifications might have been "wrong" (e.g., content a user manually un-hid was previously classified irrelevant) or how often a certain rule triggers. This could inform improving prompts or adjusting taxonomies. Because we version all data, we could even reconstruct what the system did last week vs now if needed to see improvements or regressions.

- **Security and Integrity of Explanations:** One might ask, could an advanced website detect this and try to fool it (for example, a site might notice content is being hidden and could inject misleading elements)? Since this is an early-stage project and a user-chosen filter, we assume cooperative content in the sense that we're not adversarially filtering (it's not like an ad-blocker blocking ads where sites might try to circumvent it). However, from a security standpoint, we treat the provenance data as sensitive: it should only be accessible to the user, not leaked to third parties, since it might include their interest profile. Our proxy will ensure that the injected banner and links do not expose any personal info except through the intended local channels.

- **Comparisons to End-to-End AI Filtering:** It's worth contrasting our explainability with a hypothetical alternative: using a single LLM prompt to, say, rewrite the page and remove irrelevant parts. While a big LLM could attempt that, it would be essentially impossible to explain *why* it removed a certain paragraph except by asking the LLM to narrate its reasoning (which might not be reliable or consistent). Our method, by breaking it down, yields a clear answer for each action. This is a conscious trade-off of a bit more development complexity for a lot more

clarity. As one of Dinis's reflections states, a one-shot GenAI method had "a large number of very important problems including lack of explainability," which the LETS pipeline solved by **splitting the problem and saving each step's result** [30]. Our project is a direct application of that philosophy in the domain of web content filtering.

Future Extensions and Opportunities

While the core architecture is now in place for sentiment and sports-interest filtering, there are many directions this project can grow, both technically and in terms of features:

- **Additional Filtering Dimensions:** We can introduce new filter criteria simply by adding new LLM extraction steps and corresponding persona preferences. For example, a profanity filter (detect and remove profane content), a reading level filter (hide content that's too complex or too simple), a source credibility filter (if we integrate a database of trusted vs untrusted news sources, we could flag or down-rank content from certain sites), or a fact-check filter (highlight potentially false claims – though that is a very challenging problem requiring external knowledge). Each of these could hook into the pipeline without fundamentally changing the architecture: they'd add new nodes/properties to content and persona graphs and then be considered in the matching logic.

- **Learning User Preferences Implicitly:** Right now, the persona is user-defined (explicit). In the future, the system could observe user behavior to refine the persona. For instance, if a user repeatedly unhides or clicks "show me more" on content that was initially filtered out, the system might learn that interest was actually there. Conversely, if they constantly hide content of a certain type (say they manually blacklist any celebrity news that got through), the system could add that to their persona as "dislikes celebrity news". This moves towards a **recommender system** style loop, where the system adapts over time. Our provenance data would be key for this, as it can trace what attribute caused a hide/show and correlate that with user feedback.

- **Collaboration with Content Providers:** In an ideal world, news sites and other content providers would supply semantic annotations for their content (e.g., using Schema.org metadata or JSON-LD embedded in pages indicating sentiment, topics, etc.). If such data is available, our system can ingest it directly instead of calling LLMs to infer it. The architecture is ready for that: essentially, skip step 5 if metadata already provides what we need. There's potential for partnerships – for example, a news site might officially support our filtering by providing an API to get structured content. Our approach could then focus only on the personalization aspect (matching to persona) and less on extraction. In any case, our reliance on LLMs can decrease over time as structured data becomes more prevalent or if we train smaller specialized models for certain domains.

- **Scaling Considerations:** If this service gains many users, we will need to scale the backend. The stateless design lets us add parallel workers easily. A possible bottleneck is the LLM API usage – we'd need to manage API rate limits and costs. Techniques like result-sharing across users become important: if 100 users are all filtering the BBC homepage around the same time, it would be wasteful for all 100 to independently call the LLM on the same content. We'd want a mechanism to share results (which we get naturally if they share the same S3). We might also implement a pub/sub or queue system: when new content is fetched, a single "analysis task" runs and all waiting requests for that page subscribe to the result. These are standard web caching tactics applied to our scenario.

- **Deployment Models:** The proxy approach might evolve into a full **custom browser** or an **extension**. For example, an Electron-based browser that has the filtering baked in could provide smoother integration (similar to how some privacy browsers block ads by design). We already explored a Pyodide (browser-based Python) extension for capturing content [20]; a variant could filter content. The advantage of an extension is that it could modify content after the page loads, entirely on the client side, which avoids sending user data to a server. However, doing heavy LLM analysis in the browser is not yet practical (unless using a local model, which on typical user hardware is tough for GPT-4-level quality). So a hybrid might be: extension captures text, sends it to our service for analysis, then the service returns instructions on what to hide. That's essentially a different way of implementing the pipeline where the browser does steps 1-4, then calls an API for step 5-7, then does 8 itself. We might try this later for a more privacy-preserving mode.

- **Use Cases Beyond News:** While we focused on news sites as an example (rich content, frequently updated, often more content than a user wants), this technology could apply elsewhere. Social media feeds could be filtered (imagine a personal Twitter filter that hides tweets about certain topics or with toxic sentiment). E-commerce sites could be filtered (perhaps hiding products that are outside a price range or not ethically sourced, per user settings). Even documentation or technical content could be filtered (e.g., in a large tech manual, highlight the parts relevant to your configuration). The architecture would remain similar: parse the web content to a graph, classify content, match to user preference graph, render results.

- **Combining with Content Generation:** Once you have this framework, you could also insert *generation* tasks. For instance, instead of simply hiding negative news, a user might want a *summary of only the good news*. We could generate a summary paragraph that replaces the whole news page, or a "sports digest" that replaces the sports page, composed of just the teams they like. Essentially, our pipeline could branch: after identifying relevant content, instead of showing it raw, ask an LLM to summarize or rewrite it in a personalized style. This goes into the territory of personalized newsletters or even AI commentators. However, generating content opens another set of explainability challenges (we would need provenance for facts in the summary), so it's an extension for further down the line.

In conclusion, the Web Content Filtering Project is not just about hiding a few elements on a page – it's a **framework for dynamic, personalized web experiences**. By leveraging GenAI to understand content and using graphs to make decisions explicit and traceable, we set a foundation that can be extended in myriad ways. Our current focus is delivering the promised MVP (sentiment and sports filtering) reliably and clearly. But we're also keeping an eye on the broader vision: a future where users have **complete control and insight** into the information they consume, powered by open-source tools, transparent algorithms, and AI assistance where it adds value.

Conclusion

The Web Content Filtering Project showcases a novel integration of **Generative AI, knowledge graphs, and web technology** to empower users in tailoring their online content consumption. By intercepting web pages and transforming them in real-time, we enable features like hiding unwanted content and spotlighting what matters most to an individual user – all done in a way that is explainable, deterministic, and efficient.

This technical briefing has walked through the detailed architecture: from the initial page capture and parsing, through multi-stage AI-driven analysis (sentiment detection, topic classification) using **LLMs**

with structured outputs, to the construction of semantic graphs that allow precise matching against a user's interest graph. We've emphasized how the system saves each intermediate result (following a LETS pipeline approach) to achieve transparency and reliability. This approach, influenced by Dinis Cruz's prior research on provenance in AI systems [3] [30], means that every filtered page comes not as an inscrutable magic trick, but as the end result of a series of traceable transformations – any of which can be inspected or audited. The use of **open-source tools** like OSBot and MGraph-DB is not only a practical choice but also grounds the project in a wider community of graph and AI innovation. We have built on these tools and in doing so credited the past work that made them available, from OSBot's type-safe classes for LLM JSON handling [2] to MGraph's memory-first, JSON-backed graph model [8].

The initial implementation will provide immediate user-facing value in the form of customizable news filtering, but the architecture is flexible and extensible. As we move forward, we foresee expanding the system to cover more use cases, refining the ontologies and taxonomies that drive classification, and possibly collaborating with content publishers for even richer data integration. We will also gather feedback from users and the technical team to iterate on the design – for example, fine-tuning the balance between LLM usage and direct graph logic, or improving the UI/UX of the in-browser modifications and explanations.

In summary, this project represents a step toward a more **personalized and user-centric web**, where **the user is in control of content filtering criteria** (rather than relying on each website's one-size-fits-all design), and where AI serves as a powerful assistant to implement the user's intent, but does so under the user's guidance and with full accountability. By coupling GenAI with deterministic graphs and by treating data as a first-class asset (storing everything for reuse and inspection), we deliver a system that is both cutting-edge and trustworthy.

We look forward to building this in collaboration with the team and, as we do so, continuing to document and share the lessons learned. The outcome will not just be a useful product but also a reference architecture for **GenAI-driven content personalization** that others can learn from or replicate, further crediting and building upon the open-source and research contributions (like those by Dinis Cruz) that have paved the way.

Sources Cited:

- Dinis Cruz, *Project InsightFlow: GenAI-Powered Transformation of Regulatory and News Feeds* (2025) – architecture for content/persona graphs and LLM pipeline [1] [34] [14].
- Dinis Cruz, *LETS: A Deterministic and Debuggable Data Pipeline Architecture* (2025) – concepts of saving intermediate results, provenance, MGraph, OSBot TypeSafe [17] [3] [8] [2].
- Dinis Cruz, *Web Content Capture Extension with Pyodide and Serverless Backend* (2025) – approach for capturing and hashing web content for storage [5] [6].

---

[1] [9] [10] [11] [12] [13] [14] [15] [33] [34] Project InsightFlow: GenAI-Powered Transformation of Regulatory and News Feeds - Dinis Cruz - Documents and Research
https://docs.diniscruz.ai/2025/05/03/project-insightflow__genai-powered-transformation-of-regulatory-and-news-feeds.html

[2] [3] [4] [7] [8] [16] [17] [18] [19] [22] [23] [24] [25] [26] [27] [28] [29] [30] LETS (Load, Extract, Transform, Save): A Deterministic and Debuggable Data Pipeline Architecture - Dinis Cruz - Documents and Research
https://docs.diniscruz.ai/2025/05/27/lets__load-extract-transform-save__a-deterministic-and-debuggable-data-pipeline_architecture.html

5 6 20 21 files.diniscruz.ai
https://files.diniscruz.ai/github/pdf/2025/05/18/project__web-content-capture-extension-with-pyodide-and-serverless-backend.pdf

31 32 Beyond Static Ontologies: How GenAI Powers Self-Improving Knowledge Graphs
https://www.linkedin.com/pulse/beyond-static-ontologies-how-genai-powers-knowledge-graphs-dinis-cruz-ombpe